

Отчёт по лабораторной работе №13

Дисциплина: Операционные системы

Терентеевская Светлана Александровна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
4.1	Задание 1	8
4.2	Задание 2	9
4.3	Задание 3	11
4.4	Задание 4	12
5	Выводы	15
6	Список литературы	16

Список иллюстраций

4.1	Файл searh.sh	8
4.2	Текстовый файл	9
4.3	Запуск search.sh	9
4.4	Файл sign на Си	10
4.5	Командный файл sign.sh	10
4.6	Запуск sign	11
4.7	Командный файл file_control.sh	11
4.8	Запуск file_control.sh	12
4.9	Файл arch_all.sh	13
4.10	Файл arch_week.sh	13
4.11	Запуск arch_all и arch_week	14

Список таблиц

1 Цель работы

- Изучить основы программирования в оболочке ОС UNIX.
- Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

- Реализовать командный файл, анализирующий ключи (-inputfile, -outputfile, -p, -C, -n) с помощью getopt для поиска строк по шаблону в файле.
- Написать программу на C, возвращающей код завершения (знак числа), и командный файл, анализирующий этот код через \$? с выводом соответствующего сообщения.
- Разработать командный файл, создающий N пронумерованных файлов (например, 1.tmp, 2.tmp) или удаляющий их, с передачей N через аргумент командной строки.
- Написать командный файл для архивации (через tar) всех файлов в указанной директории, а затем модифицировать его для упаковки только файлов, изменённых за последнюю неделю (используя find).

3 Теоретическое введение

В командных процессорах ОС UNIX (оболочках, таких как `bash`) программирование позволяет автоматизировать сложные задачи администрирования и обработки данных. В отличие от простых последовательных команд, скрипты используют управляющие конструкции: **ветвления** (`if-then-else`, `case`) и **циклы** (`for`, `while`, `until`), что делает их полноценным языком сценариев.

Ключевым инструментом для разбора параметров командной строки является встроенная команда `getopts`. Она позволяет обрабатывать ключи (например, `-f`, `-o`) и их аргументы, избавляя программиста от ручного анализа позиционных параметров.

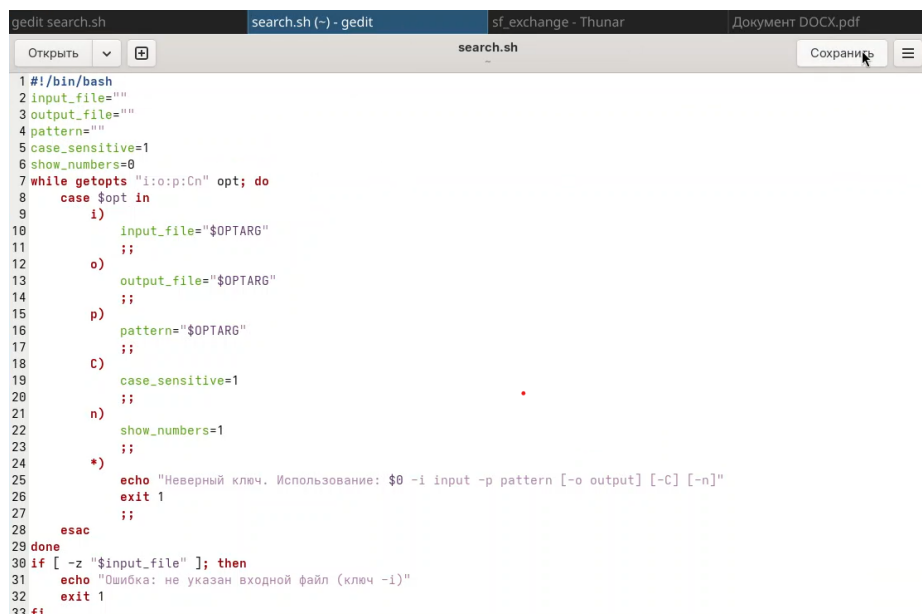
Особое место занимает **код завершения** команды (`$?`). Любая программа (включая написанные на Си) возвращает целое число: `0` — успех, ненулевое значение — ошибка или специальное состояние. Это позволяет организовывать цепочки проверок и условное выполнение.

Для работы с файлами часто используются генерация имён с помощью **мета-символов** (`*`, `?`, `[ ]`), а также команды `find` (фильтрация по времени изменения) и `tar` (архивация). Понимание различий между циклами `while` (выполняется, пока условие истинно) и `until` (выполняется, пока условие ложно) необходимо для корректной реализации повторяющихся действий.

4 Выполнение лабораторной работы

4.1 Задание 1

Используя команды `getopts` `grep`, я написала командный файл, который анализирует командную строку с ключами: `-i`inputfile, `-o`outputfile, `-p`шаблон, `-C`, `-n`, а затем ищет в указанном файле нужные строки, определяемые ключом `-p` (рис. 4.1).



```
1 #!/bin/bash
2 input_file=""
3 output_file=""
4 pattern=""
5 case_sensitive=1
6 show_numbers=0
7 while getopts "i:o:p:Cn" opt; do
8     case $opt in
9         i)
10             input_file="$OPTARG"
11             ;;
12         o)
13             output_file="$OPTARG"
14             ;;
15         p)
16             pattern="$OPTARG"
17             ;;
18         C)
19             case_sensitive=1
20             ;;
21         n)
22             show_numbers=1
23             ;;
24         *)
25             echo "Неверный ключ. Использование: $0 -i input -p pattern [-o output] [-C] [-n]"
26             exit 1
27             ;;
28     esac
29 done
30 if [ -z "$input_file" ]; then
31     echo "Ошибка: не указан входной файл (ключ -i)"
32     exit 1
33 fi
```

Рисунок 4.1: Файл search.sh

Создала текстовый файл для проверки программы (рис. 4.2).



Рисунок 4.2: Текстовый файл

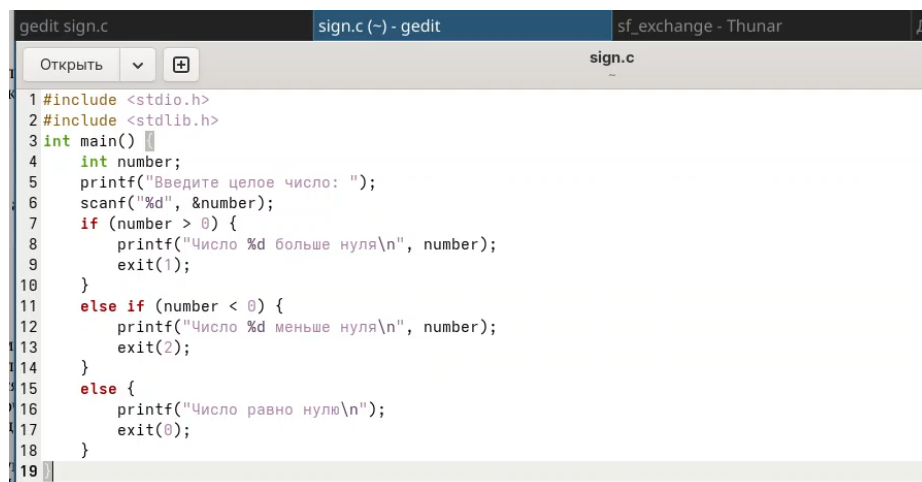
Сделала файл search.sh исполняемым и проверила его работу с разными ключами (рис. 4.3).

```
saterentevskaya@saterentevskaya:~$ chmod +x search.sh
saterentevskaya@saterentevskaya:~$ ./search.sh -i testt.txt -p hello
Hello world
HELLO world
hello world
hello WORLD
saterentevskaya@saterentevskaya:~$ gedit search.sh
saterentevskaya@saterentevskaya:~$ chmod +x search.sh
saterentevskaya@saterentevskaya:~$ ./search.sh -i testt.txt -p hello
Hello world
HELLO world
hello world
hello WORLD
saterentevskaya@saterentevskaya:~$ ./search.sh -i testt.txt -p hello -C
hello world
hello WORLD
saterentevskaya@saterentevskaya:~$ ./search.sh -i testt.txt -p Hello -C
Hello world
saterentevskaya@saterentevskaya:~$ ./search.sh -i testt.txt -p Hello -n
1:Hello world
2:HELLO world
3:hello world
4:hello WORLD
saterentevskaya@saterentevskaya:~$ |
```

Рисунок 4.3: Запуск search.sh

4.2 Задание 2

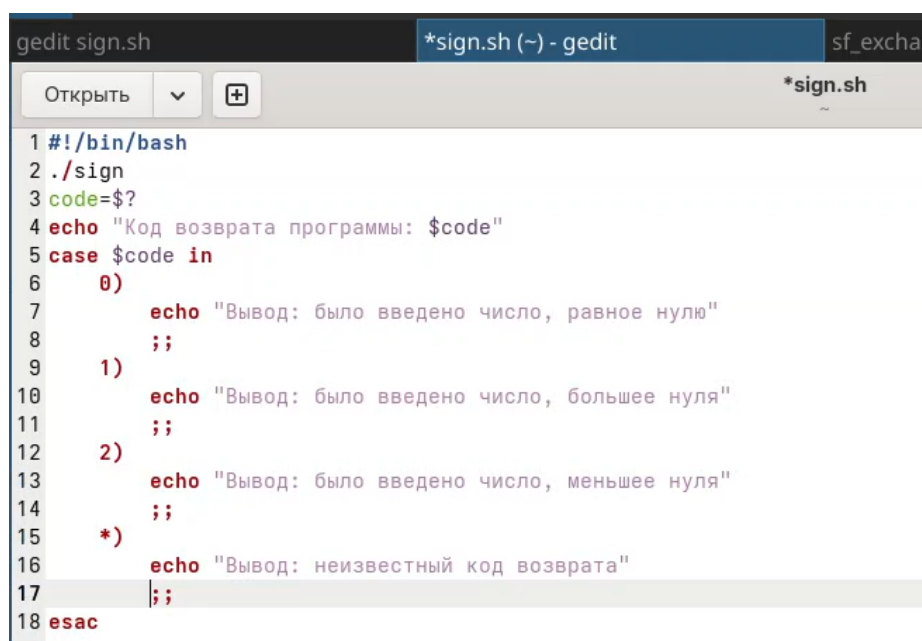
Я написала на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку (рис. 4.4).



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main() {
4     int number;
5     printf("Введите целое число: ");
6     scanf("%d", &number);
7     if (number > 0) {
8         printf("Число %d больше нуля\n", number);
9         exit(1);
10    }
11    else if (number < 0) {
12        printf("Число %d меньше нуля\n", number);
13        exit(2);
14    }
15    else {
16        printf("Число равно нулю\n");
17        exit(0);
18    }
19 }
```

Рисунок 4.4: Файл sign на Си

Дальше я написала командный файл, который вызывает эту программу и, проанализировав с помощью команды \$?, выдает сообщение о том, какое число было введено (рис. 4.5).



```
1 #!/bin/bash
2 ./sign
3 code=$?
4 echo "Код возврата программы: $code"
5 case $code in
6     0)
7         echo "Вывод: было введено число, равное нулю"
8         ;;
9     1)
10        echo "Вывод: было введено число, большее нуля"
11        ;;
12    2)
13        echo "Вывод: было введено число, меньшее нуля"
14        ;;
15    *)
16        echo "Вывод: неизвестный код возврата"
17        ;;
18 esac
```

Рисунок 4.5: Командный файл sign.sh

Сделала файл исполняемым и проверила его работу (рис. 4.6).

```

saterenteevskaya@saterenteevskaya:~$ chmod +x sign.sh
saterenteevskaya@saterenteevskaya:~$ ./sign.sh
Введите целое число: 9
Число 9 больше нуля
Код возврата программы: 1
Вывод: было введено число, большее нуля
saterenteevskaya@saterenteevskaya:~$ ./sign.sh
Введите целое число: -7
Число -7 меньше нуля
Код возврата программы: 2
Вывод: было введено число, меньшее нуля
saterenteevskaya@saterenteevskaya:~$ ./sign.sh
Введите целое число: 0
Число равно нулю
Код возврата программы: 0
Вывод: было введено число, равное нулю

```

Рисунок 4.6: Запуск sign

4.3 Задание 3

Я написала командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N . Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют) (рис. 4.7).

```

gedit file_control.sh  file_control.sh (-) - gedit  sf_exchange - Thunar  Документ DOCX.pdf
Открыть  Сохранить
1 #!/bin/bash
2 if [ $# -ne 2 ]; then
3     echo "Использование: $0 <create|delete> <N>"
4     echo " create N - создать N файлов (1.tmp, 2.tmp, ..., N.tmp)"
5     echo " delete N - удалить файлы 1.tmp ... N.tmp"
6     exit 1
7 fi
8 action="$1"
9 N="$2"
10 if ! echo "$N" | grep -qE '[0-9]+'; then
11     echo "Ошибка: N должно быть целым положительным числом"
12     exit 1
13 fi
14 case "$action" in
15     create)
16         echo "Создаю $N файлов..."
17         for i in $(seq 1 $N); do
18             filename="${i}.tmp"
19             touch "$filename"
20             echo "Создан файл: $filename"
21         done
22         echo "Готово. Создано $N файлов."
23     ;;
24     delete)
25         echo "Удаляю файлы..."
26         for i in $(seq 1 $N); do
27             filename="${i}.tmp"
28             if [ -f "$filename" ]; then
29                 rm "$filename"
30                 echo "Удалён файл: $filename"
31             else
32                 echo "Файл $filename не найден, пропускаем"
33             fi
34         done
35     ;;
36 esac

```

Рисунок 4.7: Командный файл file_control.sh

Сделала исполняемым и запустила (рис. 4.8).

```

saterenteevskaya@saterenteevskaya:~$ chmod +x file_control.sh
saterenteevskaya@saterenteevskaya:~$ ./file.control.sh create 5
bash: ./file.control.sh: Нет такого файла или каталога
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh create 5
Создаю 5 файлов...
Создан файл: 1.tmp
Создан файл: 2.tmp
Создан файл: 3.tmp
Создан файл: 4.tmp
Создан файл: 5.tmp
Готово. Создано 5 файлов.
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh delete 3
Удаляю файлы...
Удалён файл: 1.tmp
Удалён файл: 2.tmp
Удалён файл: 3.tmp
Готово.
saterenteevskaya@saterenteevskaya:~$ ls *tmp
4.tmp 5.tmp
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh delete 2
Удаляю файлы...
Файл 1.tmp не найден, пропускаем
Файл 2.tmp не найден, пропускаем
Готово.
saterenteevskaya@saterenteevskaya:~$ ./file_control.sh delete 5
Удаляю файлы...
Файл 1.tmp не найден, пропускаем
Файл 2.tmp не найден, пропускаем
Файл 3.tmp не найден, пропускаем
Удалён файл: 4.tmp
Удалён файл: 5.tmp
Готово.
saterenteevskaya@saterenteevskaya:~$ |

```

Рисунок 4.8: Запуск file_control.sh

4.4 Задание 4

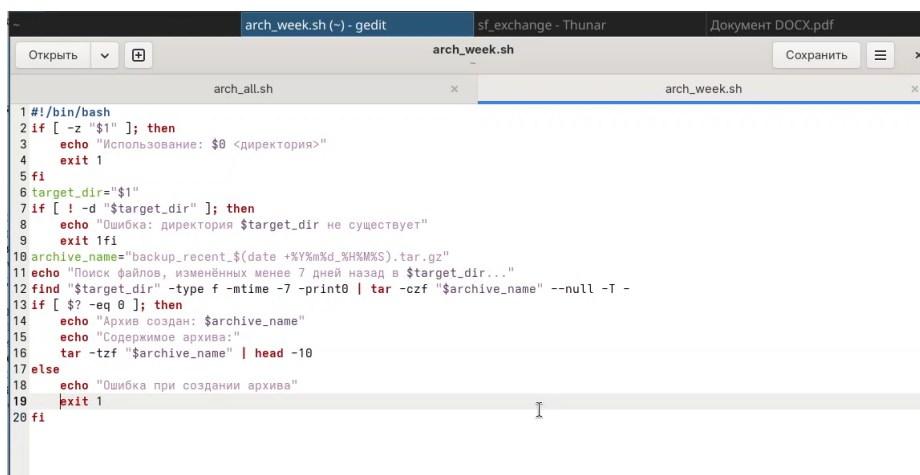
Написала командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории (рис. 4.9).



```
1 #!/bin/bash
2 if [ -z "$1" ]; then
3     echo "Использование: $0 <директория>"
4     exit 1
5 fi
6 target_dir="$1"
7 if [ ! -d "$target_dir" ]; then
8     echo "Ошибка: директория $target_dir не существует"
9     exit 1
10 fi
11 archive_name="backup_$(date +%Y%m%d_%H%M%S).tar.gz"
12 echo "Архивирую все файлы из $target_dir..."
13 tar -czf "$archive_name" -C "$target_dir" .
14 echo "Архив создан: $archive_name"
```

Рисунок 4.9: Файл arch_all.sh

Далее модифицировала его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовала команду find), создав для этого второй файл (рис. 4.10).



```
1 #!/bin/bash
2 if [ -z "$1" ]; then
3     echo "Использование: $0 <директория>"
4     exit 1
5 fi
6 target_dir="$1"
7 if [ ! -d "$target_dir" ]; then
8     echo "Ошибка: директория $target_dir не существует"
9     exit 1
10 fi
11 archive_name="backup_recent_$(date +%Y%m%d_%H%M%S).tar.gz"
12 echo "Поиск файлов, измененных менее 7 дней назад в $target_dir..."
13 find "$target_dir" -type f -mtime -7 -print0 | tar -czf "$archive_name" --null -T -
14 if [ $? -eq 0 ]; then
15     echo "Архив создан: $archive_name"
16     echo "Содержимое архива:"
17     tar -tzf "$archive_name" | head -10
18 else
19     echo "Ошибка при создании архива"
20     exit 1
21 fi
```

Рисунок 4.10: Файл arch_week.sh

Сделала файлы исполняемыми, создала тестовую директорию с файлами: для одного изменила дату, а другой не трогала и запустила (рис. 4.11).

```

saterenteevskaya@saterenteevskaya:~$
saterenteevskaya@saterenteevskaya:~$ chmod +x arch_week.sh
saterenteevskaya@saterenteevskaya:~$ chmod +x arch_all.sh
saterenteevskaya@saterenteevskaya:~$ mkdir -p test_dir
saterenteevskaya@saterenteevskaya:~$ cd test_dir
saterenteevskaya@saterenteevskaya:~/test_dir$ echo "Старое содержимое" > old_file.txt
saterenteevskaya@saterenteevskaya:~/test_dir$ touch -t 202001091120 old_file.txt
saterenteevskaya@saterenteevskaya:~/test_dir$ echo "Новое содержимое" > new_file.txt
saterenteevskaya@saterenteevskaya:~/test_dir$ ls
new_file.txt  old_file.txt
saterenteevskaya@saterenteevskaya:~/test_dir$ cd
saterenteevskaya@saterenteevskaya:~$ ./arch_all.sh test_dir
Архивируем все файлы из test_dir...
Архив создан: backup_20200412_002402.tar.gz
saterenteevskaya@saterenteevskaya:~$ ./arch_week.sh test_dir
./arch_week.sh: строка 21: syntax error: unexpected end of file from `if' command on line 7
saterenteevskaya@saterenteevskaya:~$ gedit arch_week.sh

(gedit:18643): Gtk-WARNING **: 00:26:51.506: Calling org.freedesktop.portal.Inhibit.Inhibit failed: GDBus.Error:org.freedesktop.DBus.Error.UnknownMethod: Не существует интерфейс "org.freedesktop.portal.Inhibit" на объекте по пути /org/freedesktop/portal/desktop
saterenteevskaya@saterenteevskaya:~$ chmod +x arch_week.sh
saterenteevskaya@saterenteevskaya:~$ ./arch_week.sh test_dir
Поиск файлов, измененных менее 7 дней назад в test_dir...
Архив создан: backup_recent_20200412_002723.tar.gz
Содержимое архива:
test_dir/new_file.txt
saterenteevskaya@saterenteevskaya:~$

```

Рисунок 4.11: Запуск arch_all и arch_week

5 Выводы

В ходе выполнения данной лабораторной работы я изучила основы программирования в оболочке ОС UNIX и научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

6 Список литературы

ТУИС

Методичка к лабораторной работе №13